电子信息工程训练

之

产品研发过程中的电装与调试

新智能插座DIY调试

李培弘

调试工序

- 硬件系统测试（涉及系统多模块的硬件、软件联测）

在之前的电装以及基本的硬件纠错工序完成的基础上，我们所设计的硬件已基本能安全通电运行，接着是确认各部分电路的性能参数，及其与软件测控的配合性等方面的调整测试。如：确认智能插座的硬件物理通道与软件逻辑通道的对应关系。

调试工序

- 硬件参数标定

对一些需要量化测量或控制的电路硬件，进行计量调整，并与相关软件相配合，以实现计量等方面的标准化处理。如：通过对软硬件的调整，使系统所采集到的温度值与实际当前室温对应等。

- 调试中软件与硬件相结合，以便使软件与硬件获得性能最佳的匹配。

硬件调试的准备

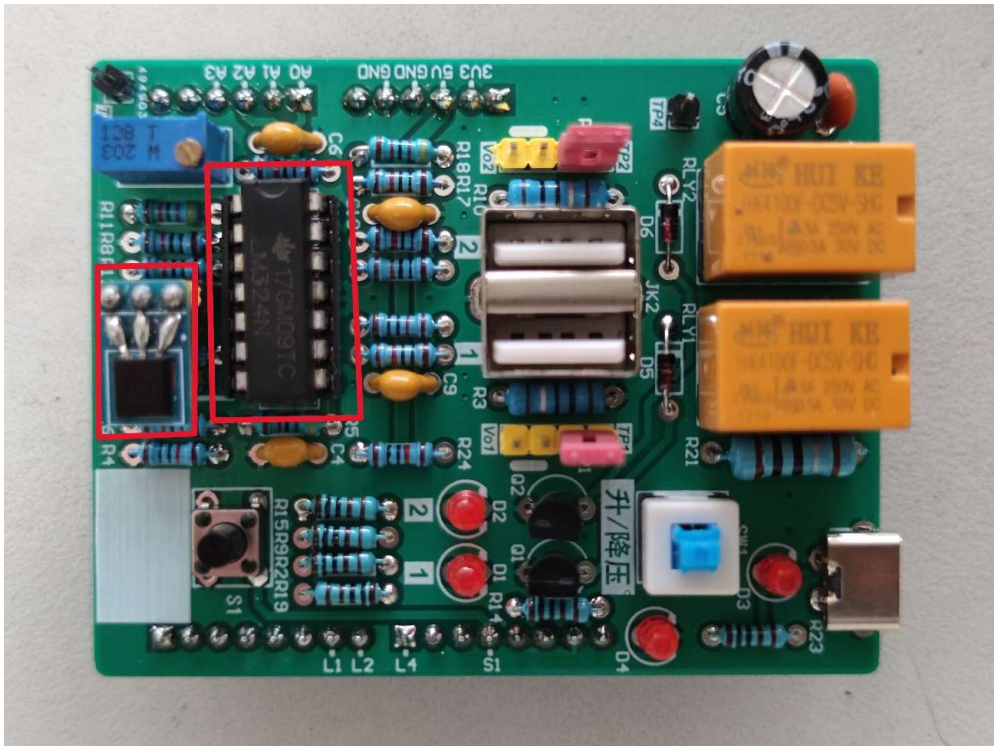
硬件组装

- 领取如下配件：LM324集成运算放大器、LM35温度传感器，以及Arduino嵌入式系统主板。
- 注意：LM324和LM35两芯片的安装有方向性！装反会导致电路剧烈发热甚至烧毁，请细心安装，并请同组成员相互检查确认！



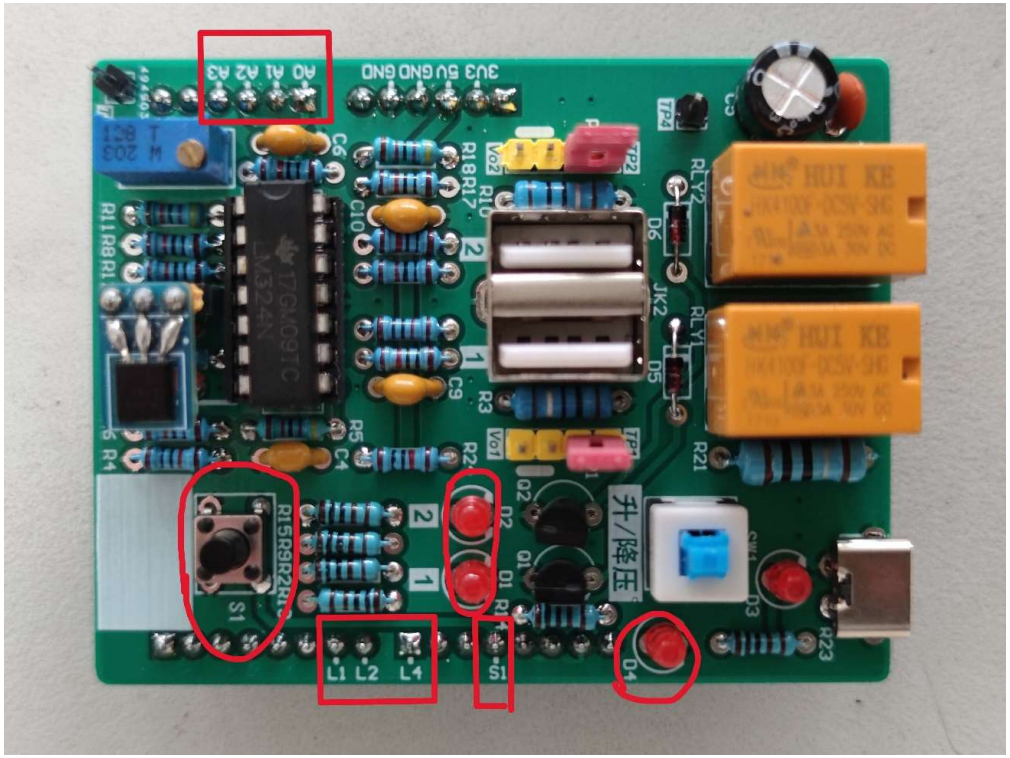
硬件组装

- 芯片LM324以及LM35安装情况如下图所示
- 安装时注意LM324和LM35的方向！



硬件控制引脚汇总

- 板子上的LED控制信号输出；按钮的状态信号输入；以及电压、电流及温度等转换获得的电压信号，都是通过各排针引脚，连接到下面主板的ESP32C3主板的CPU上。

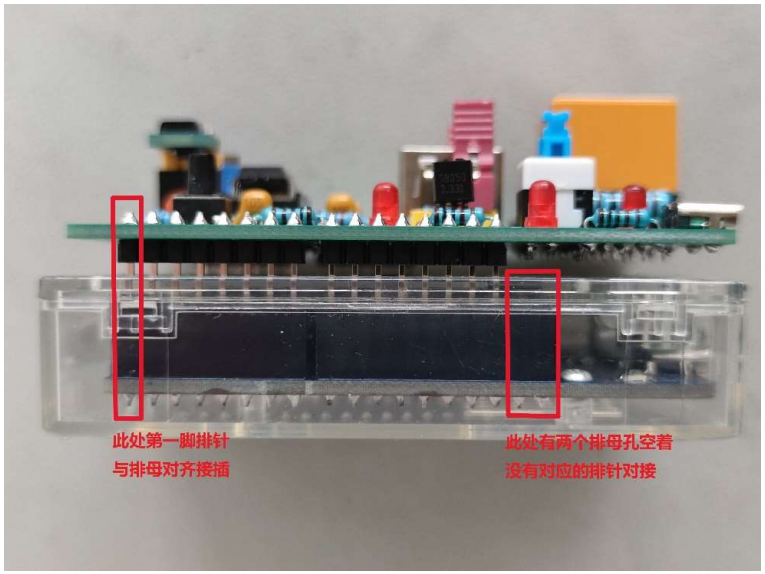


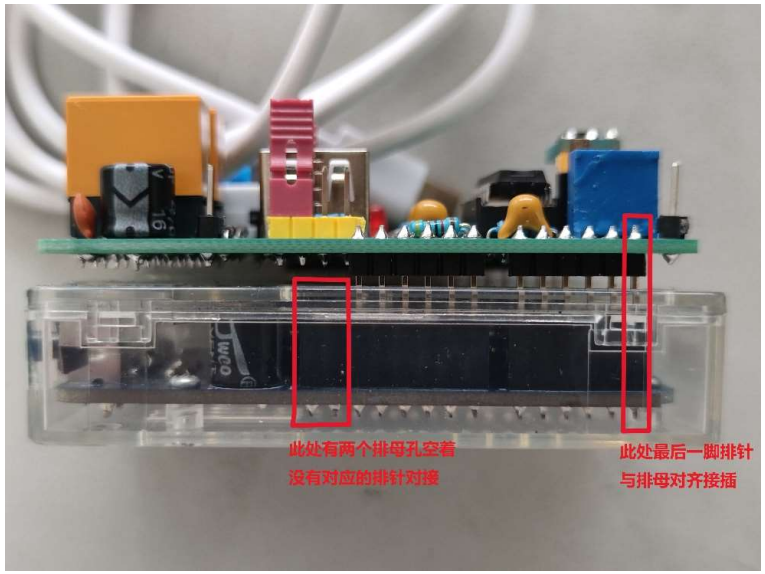
引脚对应的控制输出或采集输入关系如下：

L1<->LED D1控制输出
L2<->LED D2控制输出
L4<->LED D4控制输出
S1<->按钮S1状态输入
A0<->电流1数字化采样
A1<->电流2数字化采样
A2<->电压数字化采样
A3<->温度数字化采样

硬件组装

- 在确认上述安装无误后将你的PCB装入ESP32C3_Dev主板，如图所示。
- 注意：两侧排针引脚与主板排母孔之间不要错插！不要漏插！更不要反插！

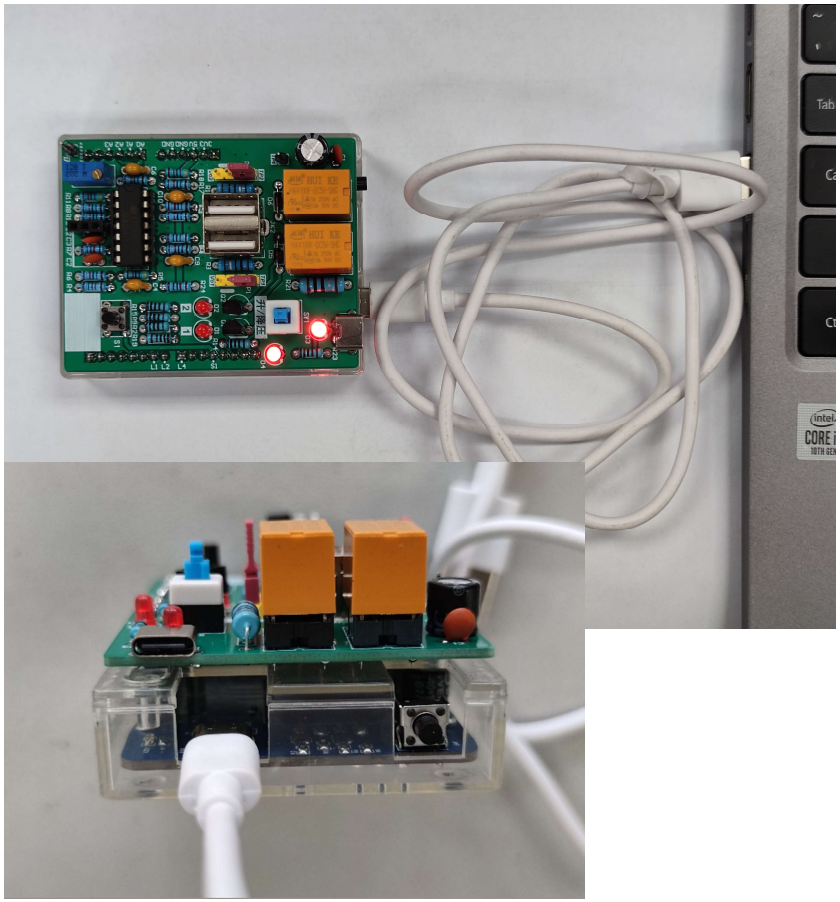




硬件组装

- 在本测试中，不要在你电装电路板的硬件系统上接外接5V稳压电源，要在ESP32C3_Dev主板的USB接口上接上USB电缆线，并与PC相连。现在你的电路板是通过PC上USB接口的5V输出来供电的。
- 注意：在如下测试中，请保持“SW1”为按下状态，即该开关为闭合状态！你知道这是为什么吗？

硬件组装



警示！注意事项：

- 不要频繁地扭曲实验所提供的各种灯具的软杆，那样容易造成灯具软杆中的导线断线！
- 避免在你智能插座的2个USB-A插座JK2上同时插上小台灯和风扇，这两者的USB插头比较厚，硬要同时插上它们有可能会导致你智能插座的USB接口JK2或小台灯/风扇的插头损坏！

测试条件

- 用电设备：

可调光台灯、小台灯、风扇



- 测试仪器：直流稳压电源、万用表、示波器







调试环境及其配置

- Arduino IDE（Integrated Development Environment，集成开发环境）

——内含编程、调试、项目管理等

- 编程语言

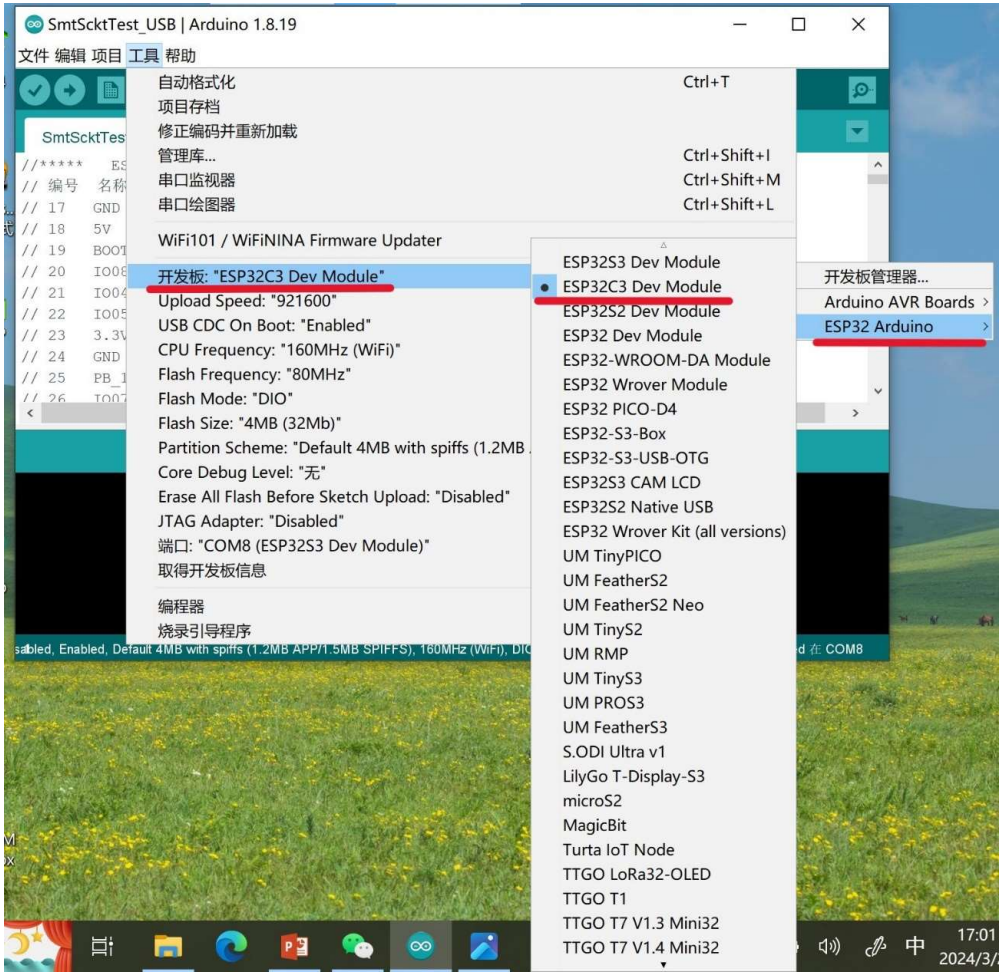
——Sketch

- 调试硬件

——串口（鉴于目前的PC已基本取消了串口硬件，可通过串口-USB接口转换，来连接PC与智能插座。（说明：ESP32C3_Dev主板上已配备了串口-USB转换硬件了，故只需一根电缆连接智能插座与PC即可在USB接口上实现串口的功能）

配置调试环境

- 打开桌面上的Arduino IDE。
- 通过菜单项“工具”选择“ESP32 Arduino” -> “ESP32C3 Dev Module”



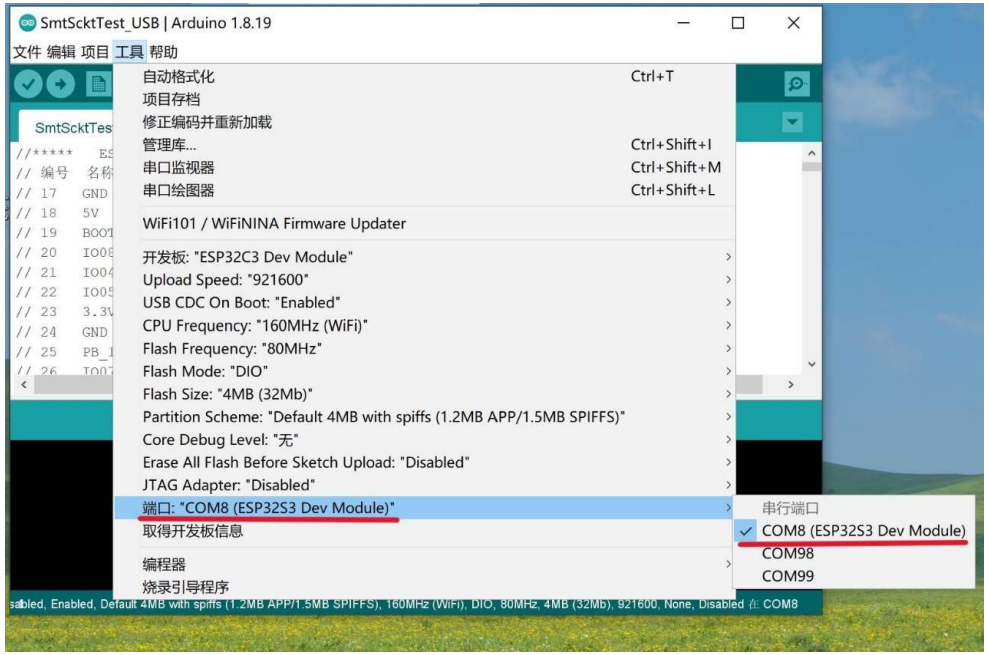
配置调试环境

- 通过菜单项“工具”配置其他ESP32C3 Dev Module的设置项如红线所示。



配置调试环境

- 在Arduino IDE软件系统中，通过“工具”->“端口”设置正确的串口（注：不一定是COM8，也可能是别的端口号COMx）。请确保该串口选项被打上“√”。
- 这样IDE就设置完成。

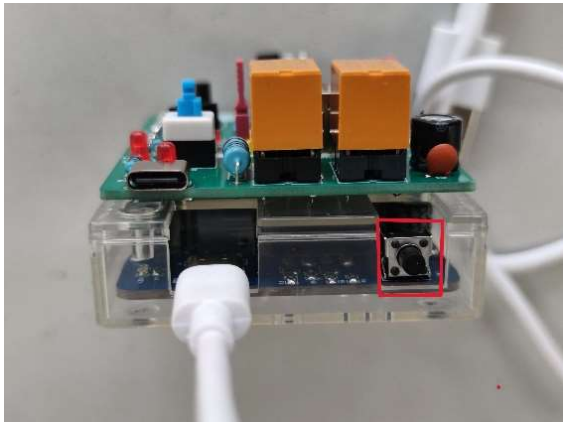


ESP32C3_Dev的2种工作模式

- 上传模式：这种模式下，ESP32C3_Dev将停止正常运行，等待Arduino IDE中的代码编译完成并上传到ESP32C3_Dev的闪存中保存，且板子掉电后该代码也不会丢失，再次上电后还能执行该代码。
- 运行模式：这种模式下，ESP32C3_Dev闪存中的代码将被直接正常运行。一般情况下，板子上电便进入正常的运行模式，并自动运行闪存中之前保存的代码。

ESP32C3_Dev
如何在2种工作模式间切换？

在板子的USB接口连通PC的状态下：



- 长按住下图中的红框内按钮保持3秒以上，板子便进入上传模式，此后可以通过Arduino IDE将编译好的程序代码上传到板子的闪存中。
- 短促地按一下下图中红框内的按钮并放开，板子便回到正常的运行模式，并自动运行前一次写入闪存的代码。

系统硬件调试

通用调试方法介绍

- 工程中常用的硬件系统通用调试方法1：

我们在硬件工程研发中，并不是一开始就加载功能完整的固件程序代码，而是先加载一些功能单一的调试小程序代码，用于硬件功能的调试，以及硬件参数的调整（即标定）。

关于调试小程序

- 在硬件调试中，一般需要确认各输入、输出的物理通道，以及它们与软件控制的对应关系。
- 确认各输入、输出通道是否有效可用，即是否受软件控制，是否有供电输出。
- 调整一些输入输出通道的控制参数（包括硬件参数调整和软件参数调整），即标定。

关于调试小程序

- 该调试小程序SmtScktTest_USB.ino代码文件在如下图位置，双击之即可在Arduino IDE中打开该源代码文件。将其上传到Arduino主板中即开始运行，运行中可通过“串口监视器”窗口查看运行结果。



关于调试小程序

- 我们的调试小程序SmtScktTest_USB实现的功能：

（1）循环采集2路电流，1路电压以及1路温度数据，并通过串口发送至PC上的“串口监视器”窗口中显示。

（2）可以在“串口监视器”中输入：

字符“A”，通过串口发送至智能插座，以闭合智能开关的1#插座；

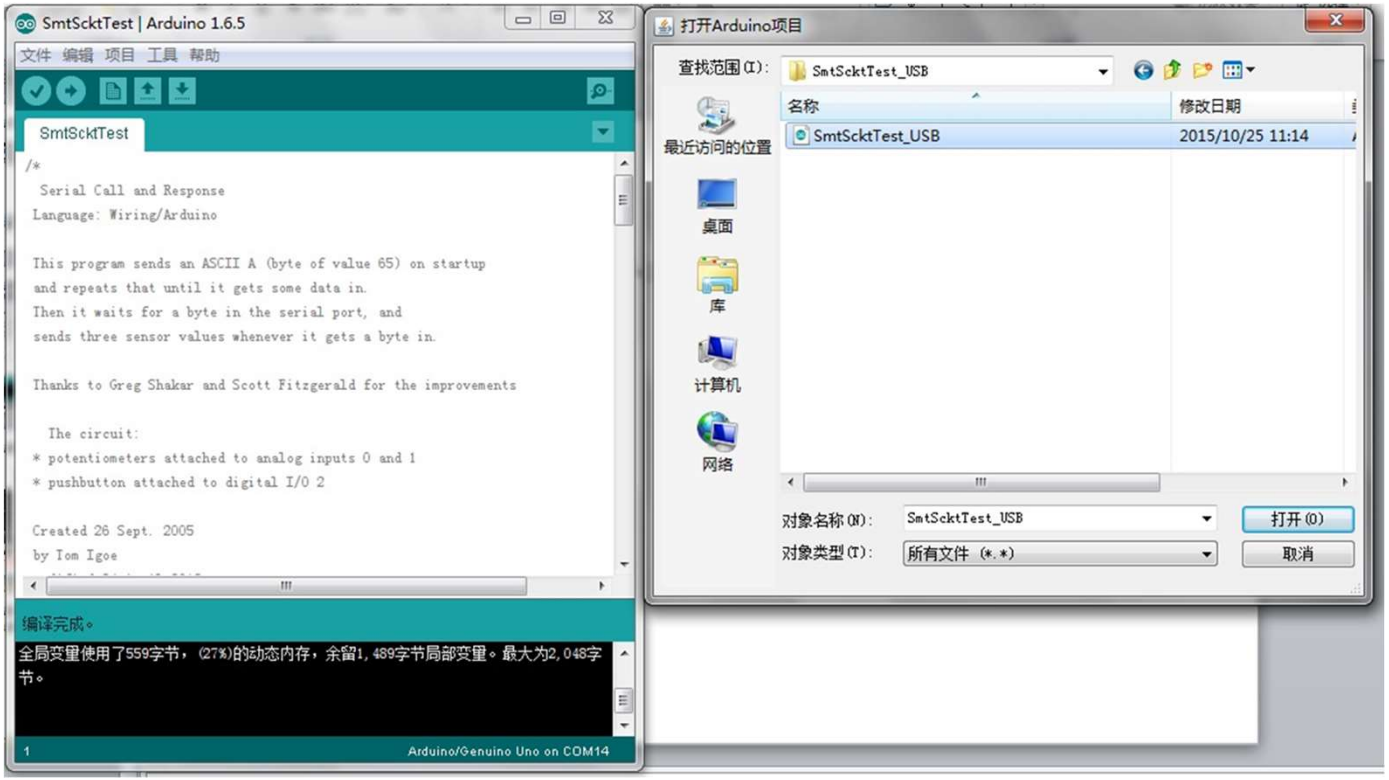
字符“a”，通过串口发送至智能插座，以开路智能开关的1#插座；

字符“B”，通过串口发送至智能插座，以闭合智能开关的2#插座；

字符“b”，通过串口发送至智能插座，以开路智能开关的2#插座。

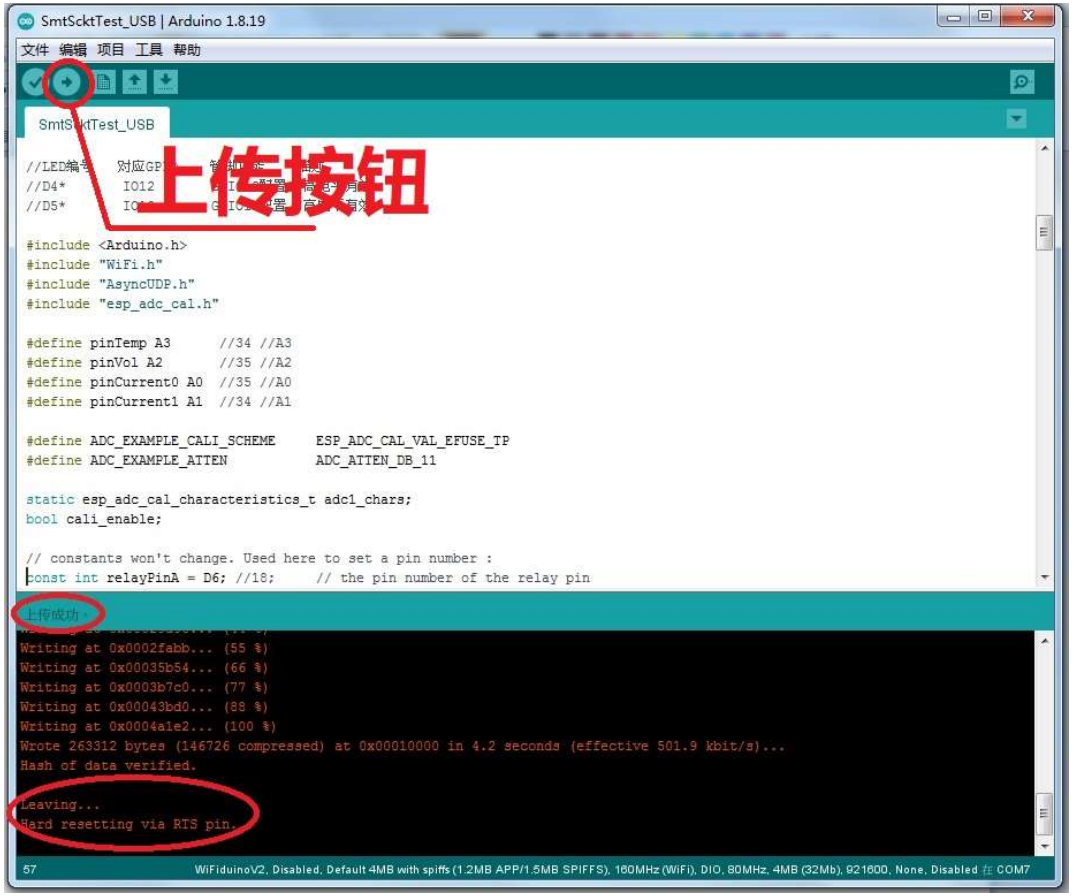
打开调试小程序

- 在你的课程下载目录里找到并双击鼠标在Arduino IDE中打开。



上传测试小程序代码

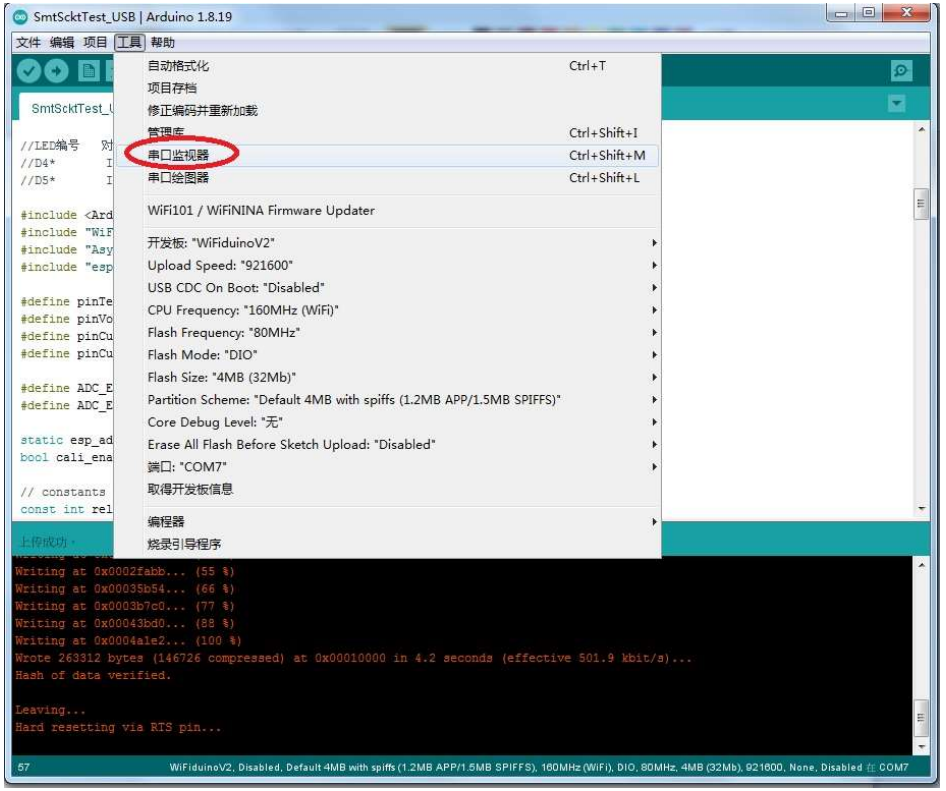
- 确保你硬件板卡ESP32C3_Dev主板的Type-C与PC的USB接口相连。上传该调试小程序代码到目标硬件：先按上传按钮，直到出现提示“上传成功”。
- 注意：如果上传代码发生错误，一般是你的端口未设对或者未预先进入上传状态。请先检查端口设置和系统运行状态！



通用调试方法介绍

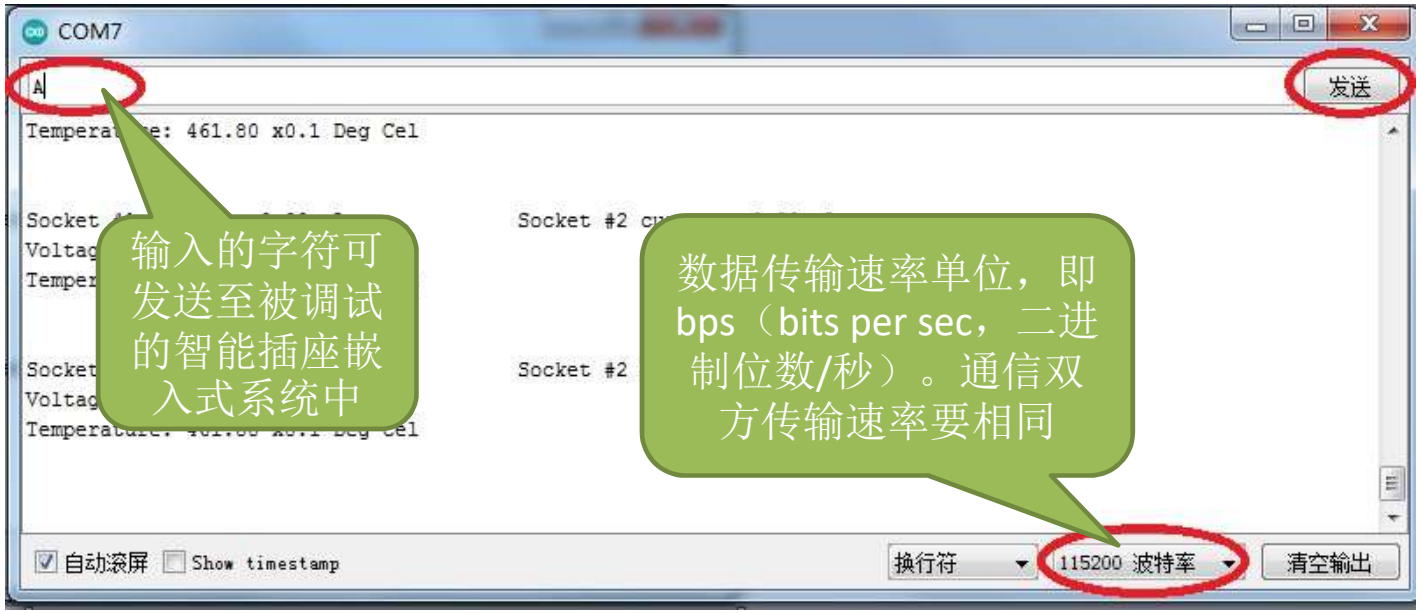
- 打开IDE中的串口监视器窗口，以便观察你硬件运行发出的结果数据。（注意串口监视器的波特率选择！）

- 通用调试方法2：串口监视器（更多场合下被称为串口终端）是一个软件功能。它是在嵌入式系统调试中的一种重要手段，即通过串口通信功能，将嵌入式系统中运行程序的结果，通过串口通信输出（也称为“抽点打印”），以便在串口终端之类的软件窗口中显示；或从这些软件中键盘输入信息并通过串口送至嵌入式系统，以实现预设的流程人工干预。



系统硬件调试开始

- 运行时“串口监视器”窗口显示如下：



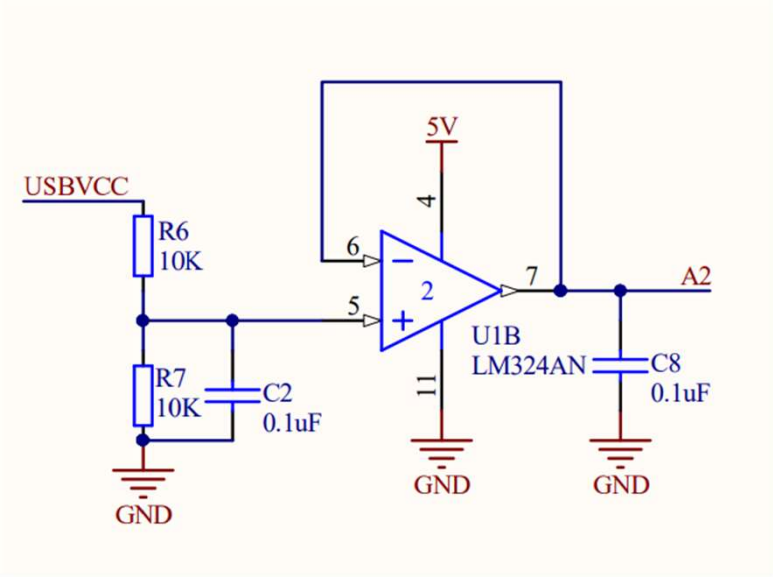
系统硬件调试开始

- 上图中的串口调试器窗口中显示的信息是从智能插座硬件的串口获得的1和2路插座的电流值，插座上的电压值，以及当前所测到的温度。我们可以利用这些信息来了解我们的硬件的基本功能是否正常工作。

系统测量的标定

基于电路计算的电压数据标定

- 通过电路计算，如下放大器电路在A2处的电压值是USBVCC的1/2。



SmtScktTest_USB | Arduino 1.8.19

文件 编辑 项目 工具 帮助

SmtScktTest_USB

```
  return val;
}

int getVoltage()
{
  uint32_t val = analog_Read(pinVol);

  //Serial.print("Voltage=");
  //Serial.println(val,DEC);

  return val*2;
}
```

基于硬件调整的温度值标定调试

- 电路中的可变电阻VR1是用于调整温度测量值输入时的转换比例的，所以你需要根据你在测试程序中显示的温度值，同时参考当前室温（可通过我们教室中安放的温度计来读取），并通过调整可变电阻VR1的螺丝旋钮来调整显示值，并使之与实际室温的尽量保持一致（正常测试环境下，如果存在少许的误差可能是元器件的参数离散性导致，请你在提交的报告中加以备注）。



在报告中完成下表填写。⑥

- 注意：如果误差很大，有可能是你电装中发生错误（一般是元器件装错位置了），请自行仔细检查！

项目	测量值
当前实际室温（摄氏度）	31.4
经你完成调试后测试程序显示温度（摄氏度）	31.520
是存在严重的元器件离散性问题？（是/否）	**否**

基于软件后处理的电流数据标定

- 电流标定

I = k * val + b

val为电流采样值；

I为经过线性矫正后的电流标定值；

k、b为标定系数。

注意：最初代码中k=1，b=0。

```
void loop()
{
  // read analog inputs:
  Sensor0 = getCurrent(0);
  Sensor1 = getCurrent(1);
  Sensor2 = getVoltage();
  Sensor3 = getTemperature();

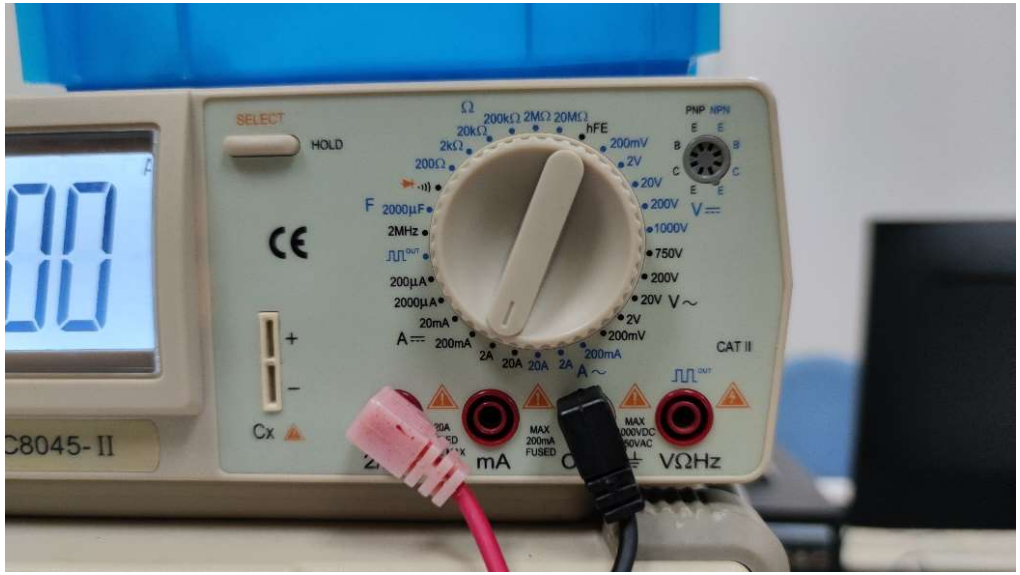
  // send sensor values:
  Serial.print("Socket #1 current: ");
  Serial.print((float)Sensor0*1+0);
  Serial.print(" mA");
  Serial.print("\t\t");
  Serial.print("Socket #2 current: ");
  Serial.print((float)Sensor1*1+0);
  Serial.println(" mA");
```

基于软件后处理的电流数据标定

- 标定方法：测量 2 组电流值，即用电流表测量每组电流的实际读数，同时在串口监视器中读取各组对应的电流采样值。
- 代入I = k * val + b即可求解k和b，即可求得标定参数公式。
- 将上述公式编程加入软件处理即可完成实时的采样标定处理，这时串口监视器上显示的电流值即和实际值相一致了。

提示：电流的测量

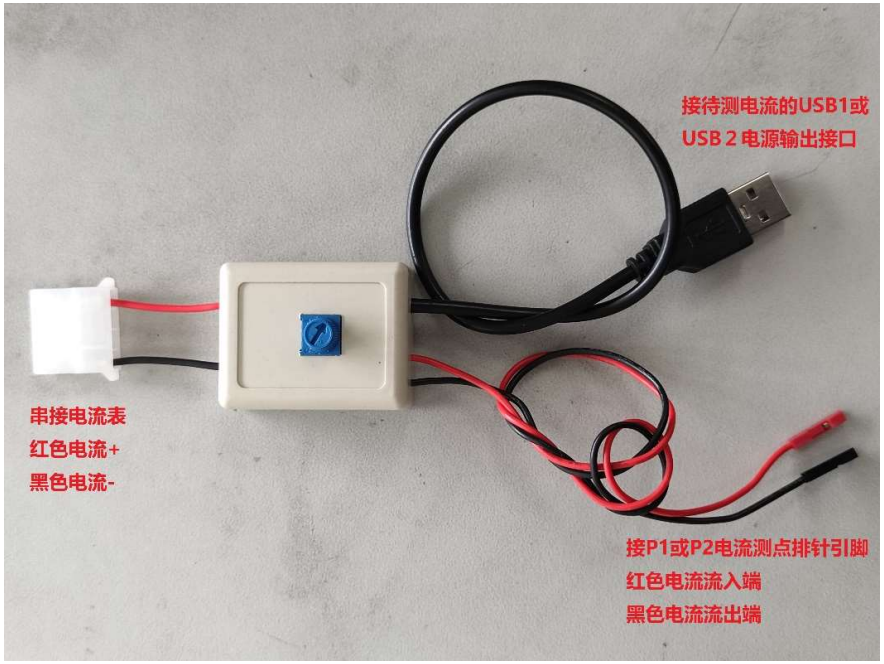
- 注意电流测量时万用表的电流量程和表棒接口



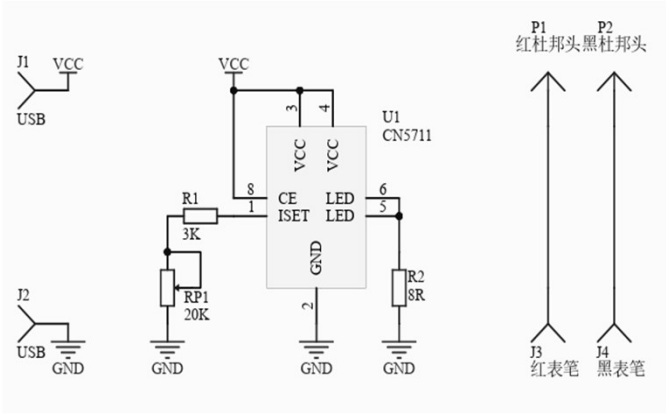
电子负载——用于电流标定

- 工程中常用的硬件系统通用调试方法3：

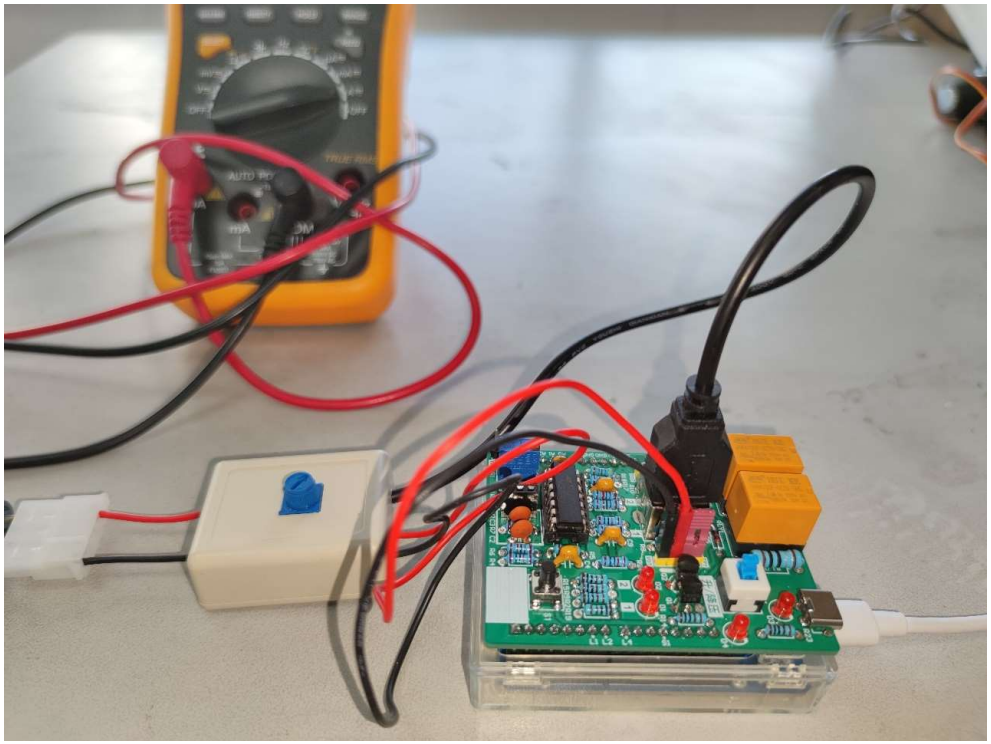
通过自制一些小硬件来提高研发中，甚至量产后的工作、生产效率。下图便是一个自制的电子负载。



盒中电路如下图，你可以把它看成一个通过可变电阻调整的电阻负载



提示：电流的测点设置



- 移动跳线帽，将P1或P2转换成电流测量模式，并将电流表和电子负载，转动旋钮即可改变流过它的电流。

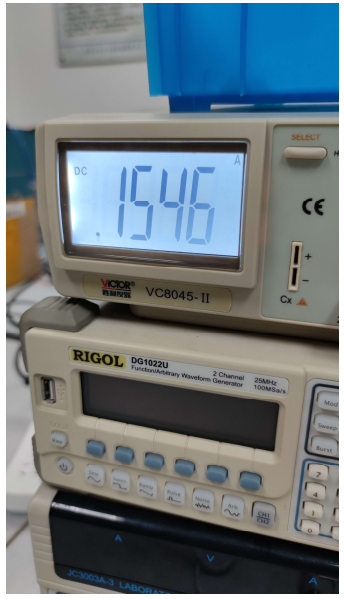
提示：电流测量时的接线

USB1电流标定接法



USB2电流标定接法





这样我们便可从电流表I以及小程序Val的显示，分别获得电流实际值和小程序测量值，从而完成标定参数k和b的计算。

基于软件后处理的电流数据标定

完成如下针对电流测量的软件标定步骤，并将标定过程记录于小结报告中。⑦

（1）采样电流数据填写在下表中：

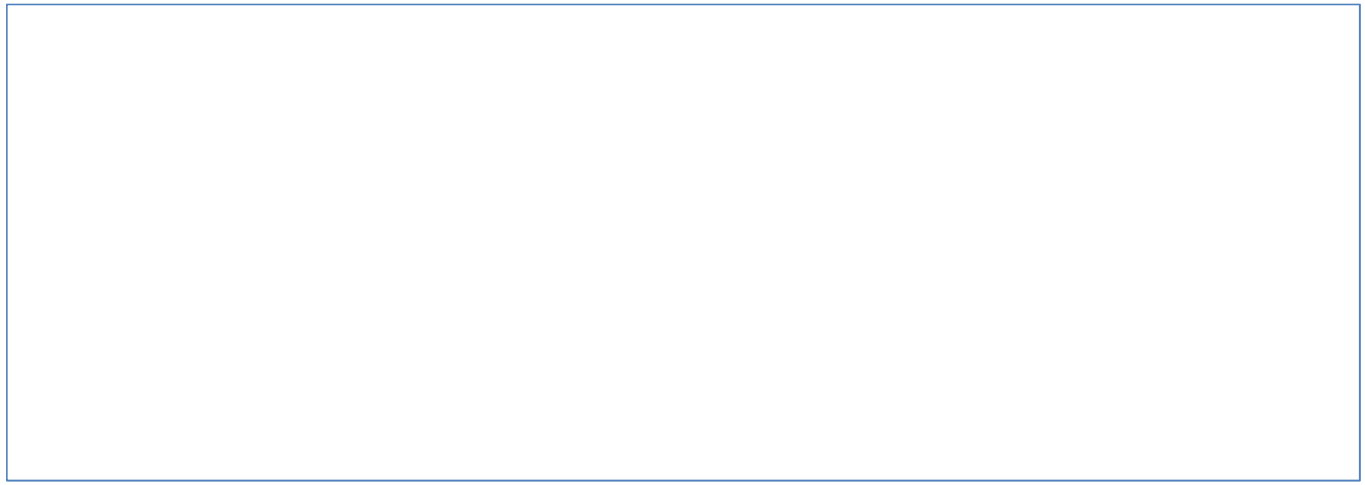
Val（串口监视器读取电流值）	I（P1、2处测得的实际电流值）
483mA	80.8mA
2700mA	467.1mA

（2）计算求解k、b，最终的标定公式为：

$$I = 0.174Val - 3.242$$

基于软件后处理的电流数据标定

（3）根据计算结果，修改后的代码段截屏如下：



基于软件后处理的电流数据标定

（4）用你标定好的小程序，采集量程范围内的10个值，并比较其与电流表实际读数的差异。如果出现较大的误差，需分析原因并重新标定。

Val（串口监视器读取电流值）	I（P1、2处测得的实际电流值）	误差

基于软件后处理的电流数据标定

（5）上面所述仅完成了一路电流的标定。另一路的电流标定参数也一样吗？如果不一样，仿照上面所述自行完成之。

软硬件相结合的系统调试

测试任务一

- 利用小台灯和小程序SmtScktTest_USB，确认程序控制USB插座中1#和2#插座的分别由哪个控制引脚控制，并完成下列各测试任务表。
- 提示：本实验需要读懂下列相关的程序段：

```
case 'a':
  digitalWrite(relayPin10,LOW);
  break;
case 'A':
  digitalWrite(relayPin10,HIGH);
  break;
case 'b':
  digitalWrite(relayPin9,LOW);
  break;
case 'B':
  digitalWrite(relayPin9,HIGH);
  break;
```

测试任务一

在完成上述测试后，完成下表填写:⑧

输出命令	1#和2#插座的状态（通/断）	控制电压（用电压表测量L1或L2）
发送字符A	智能插座1#插座的输出电压，P1的电压：5.096 V	J3的 L1（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：2.944 V
发送字符a	智能插座1#插座的输出电压，P1的电压：0.001 V	J3的 L1（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：0.001 V
发送字符B	智能插座2#插座的输出电压，P2的电压：5.093 V	J3的 L2（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：2.941 V
发送字符b	智能插座2#插座的输出电压，P2的电压：0.001 V	J3的 L2（选L1或L2，即你程序中输出控制信号的那个引脚）脚电压：0.000 V

要求对你的PCB拍照（建议你在断电时进行拍摄，以防误操作导致短路烧毁板子），在照片上标出上述实验中的测点位置，并将该照片粘贴在最终报告中提交。

测试任务二

利用各类用电器，测量其在智能插座使用中的各类测量值（注意：是通过我们的测试小程序来读取下述测量值）：⑨

项目	测量值	单位
1#插座外接的用电器名称	小台灯	
1#插座电流值	187.46	mA
2#插座外接的用电器名称	可调光台灯	
2#插座电流值（最亮时）	260.19	mA
2#插座电流值（最暗时）	41.18	mA
插座电压值	5.054	V
温度测量值（环境温度）	31.520	°C
温度测量值（手指触碰温度）	33.321	°C

测试任务三

将**2#**插座上的可调光台灯换成风扇，并接通风扇的插座电源，测量风扇的各档工作电流值。（注意：是通过我们的测试小程序来读取下述测量值）：⑩

项目	测量值	单位
风扇慢速档电流值	203.44	mA
风扇中速档电流值	317.58	mA
风扇快速档电流值	380.08	mA

上述测试任务二、三中有哪些测量值有异常？利用示波器观察并拍照记录出现这些异常时输出供电电压的波形，说明其产生的原因。⑪

如何利用示波器捕获随机信号？

- 按前期训练过的针对周期信号的设定来示波器，会较难对按钮输入这类随机信号的准确捕获。

具体做法：用示波器CH1测量，选择触发信源为“CH1”，触发方式为“自动”，调节触发电平“LEVEL”使得波形能稳定显示。调节相应的量程旋钮“VOLTS/DIV”，和扫描周期旋钮“TIME/DIV”使得波形显示大小合适。记录到的波形只有按钮按下或松开后，呈稳定状态的高或低电平，变化瞬间的信号过程无法被稳定地记录。

如何利用示波器捕获随机信号？

- 换用普通触发方式来捕获，可实现对触发信号的准确冻结，以便事后对冻结的波形观察分析。

具体做法：用示波器CH1测量；选择触发信源为“CH1”，触发方式为“普通”，触发类型为“边沿触发”；触发耦合为“DC”，触发抑制为“关”，触发噪声抑制为“关”。调节触发电平“LEVEL”，使得按下输入按钮时，能触发记录波形。触发斜率设置为“下降沿”或“上升沿”，可用来更方便地捕获按下或放开按钮后的随机毛刺波形。

（乙）教学班内容

硬件调试任务一：
各放大器电路的运行参数记录

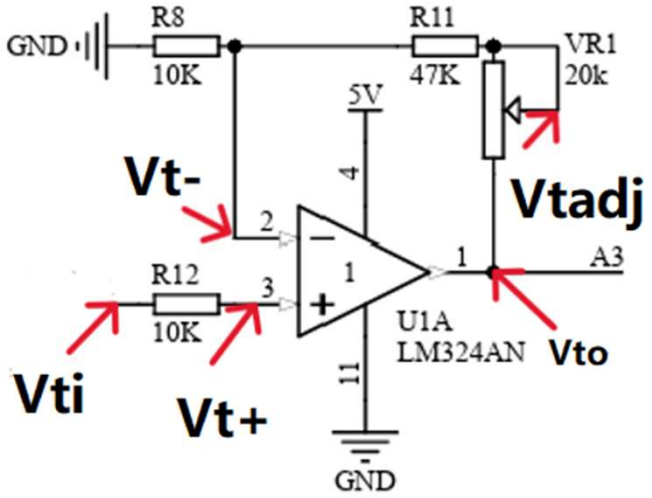
注：电子工程训练（甲）不进行本项内容的实训

- 测量各放大器的电路运行参数，并记录于下列四个表中，以便最后评价各放大器的硬件电装调试质量。
- 要求对你的**PCB**拍照（建议你在断电时进行拍摄，以防误操作导致短路烧毁板子），在照片上标出上述实验中的测点位置，并将该照片粘贴在最终报告中提交。

在总结报告上记录下面这**4**个测试结果表格⑿-乙**1**

各放大器电路的运行参数记录

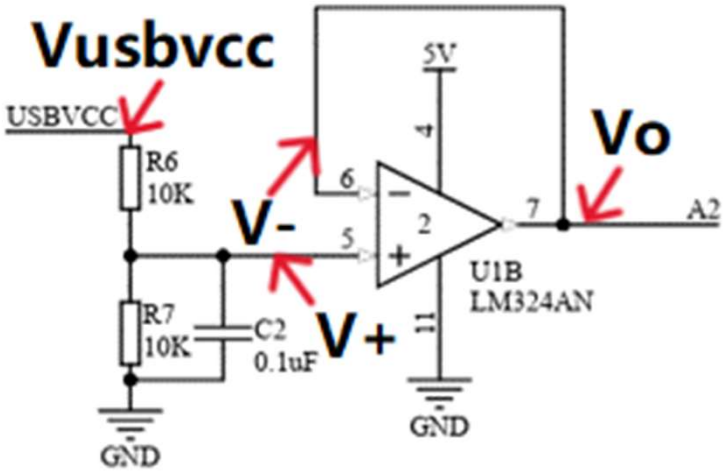
温度放大器电路运行参数	
Vti (V)	
Vtadj (V)	
Vt+ (V)	
Vt- (V)	
Vto (V)	



- 测量时SW1保持按下状态；
- 利用SmtScktTest_USB.ino关闭供电插座1、2的供电

各放大器电路的运行参数记录

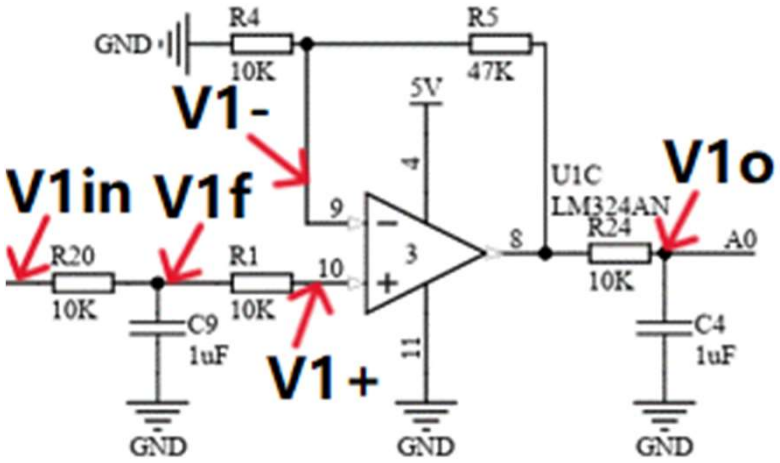
电压放大器电路运行参数	
Vusbvcc (V)	
V+ (V)	
V- (V)	
Vo (V)	



- 测量时SW1保持按下状态；
- 利用SmtScktTest_USB.ino关闭供电插座1、2的供电。

各放大器电路的运行参数记录

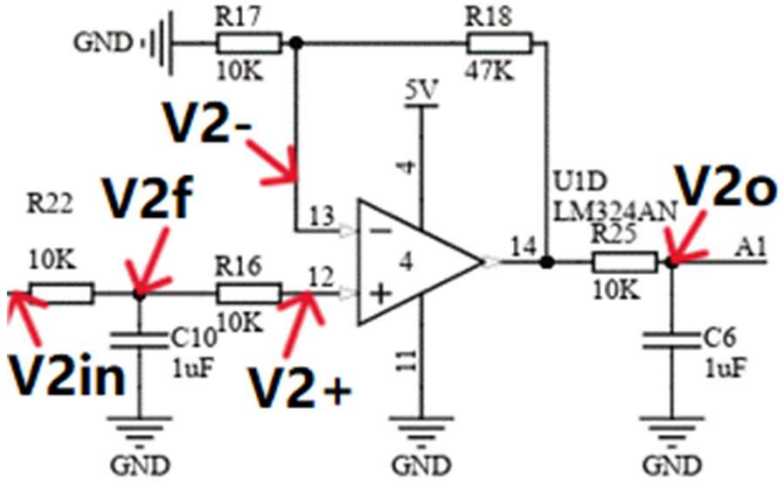
USB1电流测量放大器电路运行参数	
V1in (V)	
V1f (V)	
V1+ (V)	
V1- (V)	
V1o (V)	



- 测量时SW1保持按下状态，供电插座1连接小台灯；
- 利用SmtScktTest_USB.ino打开供电插座1，关闭供电插座2的供电。

各放大器电路的运行参数记录

USB2电流测量放大器电路运行参数	
V2in (V)	
V2f (V)	
V2+ (V)	
V2- (V)	
V2o (V)	



- 测量时SW1保持按下状态，供电插座2连接小台灯；
- 利用SmtScktTest_USB.ino打开供电插座2，关闭供电插座1的供电。

硬件调试任务二：利用示波器记录按钮S1在按下与放开瞬间的波形

- 利用示波器探头测量排针J4的第四脚（标记为S1）上的波形。要求手机拍下按钮S1被按下与放开瞬间示波器记录的波形图，并在报告中提交。

在总结报告上记录上述测量波形图⑿-乙2

软硬件联调任务：
按钮S1输入信息的软件获取

- 课程已提供了一个S1（注意：不是SW1）按钮信号读取程序key_in.ino，建议你用该程序来探究按钮被按下或放开时，按钮输出的电平信号变化。
- 该程序运行时会记录按钮变化次数，即包括按下和松开（算作两次变化）的总次数，并在串口监视器输入'r'命令后，打印出之前已记录的按钮变化总次数。在串口监视器输入'0'命令后，会将之前记录的按钮信号变化记录次数清零，并打印出清零结果。在串口监视器输入'k'命令后，会显示当前按钮的读入值，你可以按下或松开按钮的同时输入该命令来读取。
- 请先读懂理解该程序。

软硬件联调任务：
按钮S1输入信息的软件获取

- 步骤1：你需要首先弄清按下按钮/松开按钮时，Arduino程序分别读取到的按键状态值（二进制的0/1）。并记录在下表中。

检测项目	检测结果	s1按钮的程序读取值(0/1)
电路上电，按下S1按钮并保持，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平____V。 这个就是你按下按钮状态下，系统能通过软件检测到的信号。	
电路上电，松开S1按钮，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平____V。 这个就是你松开按钮状态下，系统能通过软件检测到的信号。	

在总结报告上记录上述测试结果表格⑿-乙3

软硬件联调任务：
按钮S1输入信息的软件获取

- 你可以在按钮保持在按下或松开时，按下’k’命令，以便确定按钮被按下时，被Arduino程序key_in.ino读取到的按钮状态值值是0还是1？同样，在按钮被放开时，确定被读取到的是0还是1？记录到上表中。

（甲）教学班内容

软硬件相结合实现测控功能

注：电子工程训练（乙）不进行本项内容的实训

要求说明：

- 各小组必须从“软硬件联调任务一”与“软硬件联调任务二”中，选择完成一题。从“软硬件联调任务三”与“软硬件联调任务四”中，选择完成另一题，共需完成2题 。
- 完成的任务需拍摄一段视频以展示其实现的功能效果，并通过微信公众号“东四425”中的上传功能提交。该微信公众号的功能使用参见“学在浙大”课程中“写在前面”章节的“微信公众号上传作业视频的说明”。上传时注意选择正确的“智能插座视频1~4”选项来上传。
- 视频文件命名：周x上(或下)午-421(或420)-第x组-任务一（或任务二或任务三或任务四）。视频格式为mp4或mov，分辨率不要高于720p，文件时长尽可能短小，最好有语音介绍。

（甲）教学班内容

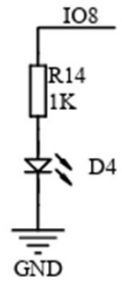
提示：你可能会用到的编程函数

（1）延时可用函数语句：

```
delay(2323);      //其中2323表示2323ms
delayMicroseconds(23); //其中23表示23us
```

（2）输出函数语句：

```
const int pinLED = 7;
digitalWrite(pinLED,HIGH); //D4亮
digitalWrite(pinLED,LOW); //D4灭
```



（3）输入函数语句：

```
const int keyPin11 = 8;
int value=digitalRead(keyPin11); //value即为引脚11上的电平值：1=高电平，0=低电平
```



提示：你可能会用到的编程函数

（4）micros()

读取自该Arduino本次启动运行以来经历的微秒（us）数，4us的分辨率，约每70min后因溢出而归零。

（5）millis()

读取自该Arduino本次启动运行以来经历的毫秒（ms）数，1ms的分辨率，约每50天后因溢出而归零。

（甲）教学班内容

软硬件联调任务一

- 设计要求（难度系数=1）：

在我们的智能插座给风扇供电时，会出现电流值不停地大小波动变化的现象，这是由于该风扇的调速采用了PWM的控制方式（想了解的同学可自行百度“PWM”学习相关知识）。请在SmtScktTest_USB.ino程序中添加一个子程序，通过调用该子程序，实现对采集的电流值取平均值，以便使同一转速挡位的电流值显示趋于平稳。

（甲）教学班内容

软硬件联调任务一

- 提示：

在取平均值时，可以选择给以往值和当前值取不同的权值，即加权平均法；也可以选择移动平均法等，来调整平均值的计算效果，以达到既能在风扇档位变化时，尽快地响应电流的变化，同时又能在各档位保持不变时，使电流的显示值趋于平稳。

（甲）教学班内容

软硬件联调任务二

设计要求（难度系数=2）：

- 利用风扇和你智能插座上的板载温度传感器，在课程提供的实例程序SmtScktTest_USB.ino的基础上改写代码，使你的风扇能根据你设置的温度期望值自动启停，以便控制温度传感器周围的温度。
- 完成代码改写，并根据当前的室温，设置你的测控温度期望值，以便展示你风扇在你设计的程序代码控制下多次自动启停以控制恒温的性能效果。提交一段视频展示你的设计效果。

（甲）教学班内容

软硬件联调任务二提示

记录测试过程中的测控数据，一般包括你设置的期望温度，测控过程中的最高温度、最低温度、风扇从启动时刻至达到设定期望值所用时间、或者风扇从关闭时刻至重新启动时刻间的时间...等等（不一定要求上述全部）你认为有展示系统性能的测试数据指标。⑿-甲1

注意：因为我们提供的风扇每次许按下相应的启动档按钮才会再次启动，所以在你进行本功能的测试期间请保持相应的按钮处于按下状态（你觉得按着哪档的按钮更好？）。

（甲）教学班内容

软硬件联调任务二提示

- 拍摄该任务的演示执行时，你可用你的风扇吹你的智能插座上的温度传感器，通过在你的软件中设置特定的温控值，使风扇时开时停，即风扇开启时温度会低于你设定的控温点，风扇关闭时温度又会高于你设定的控温点。这样就能连续演示温度利用风扇控制温度传感器周围的情况。

（甲）教学班内容

软硬件联调任务三

- 设计要求（难度系数=1）：

检测智能插座的输出供电电压，当电压高于某个设定值时，利用LED告警提示电压异常。

软硬件联调任务三的实现方案

- 建议通过修改我们提供的Arduino小程序SmtSckt Test_USB.ino来实现上述编程任务。
- 利用该程序的电压采集功能，监测智能插座的输出供电电压变化，当电压突然下降到某一预设的阈值时（该阈值可自行根据测试时的现场情况来指定），将D4发光二极管点亮；当电压恢复正常时，将D4发光二极管熄灭。

- 通用调试方法3：用板载的LED来显示你程序执行状态。你知道通过前面所述的抽点打印与上述LED显示这两种方法来显示程序运行状态，有什么不同吗？

（甲）教学班内容

软硬件联调任务三提示

- “供电电压突然下降”的情况，可以利用SW1（注意：不是S1）开关和小台灯来模拟实现。即当按下SW1开关，智能插座的输出供电电压会下降；反之，会上升（即恢复正常）。
- 特别需要注意的时，这样来实现模拟“供电电压突然下降”时，小台灯必须处于点亮状态。这是为什么？

（甲）教学班内容

软硬件联调任务四

- 设计要求（难度系数=2）：

程序代码在读取按键输入时会遇到按键信号抖动的干扰。你需要：

（1）通过设计测试试验，展示并记录这种干扰出现的场景和干扰信号的表现；

（2）通过Arduino代码的编程设计，去除这种干扰对按键信号读取的影响；

注意你在去除干扰时，要尽可能提高你所设计的Arduino代码对按键动作的响应速度。

软硬件联调任务四的实现方案

- 课程已提供了一个S1（注意：不是SW1）按钮信号读取程序key_in.ino，建议你用该程序来探究按钮被按下或放开时，按钮输出的电平信号变化；然后在此基础上修改代码，以便按钮信号读取程序实现的必要的信号读取处理。
- 该程序运行时会记录按钮变化次数，即包括按下和松开（算作两次变化）的总次数，并在串口监视器输入'r'命令后，打印出之前已记录的按钮变化总次数。在串口监视器输入'0'命令后，会将之前记录的按钮信号变化记录次数清零，并打印出清零结果。在串口监视器输入'k'命令后，会显示当前按钮的读入值，你可以按下或松开按钮的同时输入该命令来读取。
- 请先读懂理解该程序。

（甲）教学班内容

软硬件联调任务四的实现方案

- 步骤1：你需要首先弄清按下按钮/松开按钮时，Arduino程序分别读取到的按键状态值（二进制的0/1）。并记录在下表中。

检测项目	检测结果	s1按钮的程序读取值(0/1)
电路上电，按下S1按钮并保持，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平0.000V。 这个就是你按下按钮状态下，系统能通过软件检测到的信号。	0
电路上电，松开S1按钮，测量S1（J4的4脚）电平（即该点的对地电压）：	S1脚的电平3.334V。 这个就是你松开按钮状态下，系统能通过软件检测到的信号。	1

在总结报告上记录上述测试结果表格⑿-甲2

软硬件联调任务四提示

- 步骤1：你可以在按钮保持在按下或松开时，按下’k’命令，以便确定按钮被按下时，被Arduino程序key_in.ino读取到的按钮状态值值是0还是1？同样，在按钮被放开时，确定被读取到的是0还是1？记录到上表中。

（甲）教学班内容

软硬件联调任务四的实现方案

- 步骤2：需要你采用各种按键速度和连续按键次数，测试程序是否能每次都正确记录按钮次数？如果不是（即存在误读次数），可通过示波器查看并记录按钮按动时的波形，并分析误读产生的原因，写入报告中。
- 步骤3：修改该程序key_in.ino，使修改后的程序能准确地读取并记录下以各种按键速度连续按键的次数（注：每“按下-放开”只能算一次按键）。

在总结报告中附上你的这些测试屏显内容截图和必要的测试过程说明。⑿-甲3

软硬件联调任务四提示

- 我们用程序记录按键次数，并在响应命令'r'时再显示输出，是因为如果每次按键时就显示按键次数的增加，会影响系统对快速连续按键的响应性能。
- 针对慢速按键：先用'0'命令清除计数，缓慢地按下s1，接着缓慢地松开S1，这样多做几次，同时自己计数按动-松开的总次数。然后用'r'命令显示软件对这次操作记录的按键次数值。该软件计数正确吗？分析原因。
- 针对快速按键：先用'0'命令清除计数，快速地连续按下-松开S1多次后停止，并自己计数按动-松开的总次数。然后用'r'命令显示软件对这些按键操作的所记录的按键次数值。该软件计数正确吗？分析原因。

分析原因（包括示波器显示的波形）须写在总结报告中。⑿-甲4

（甲）教学班内容

软硬件联调任务四的提示

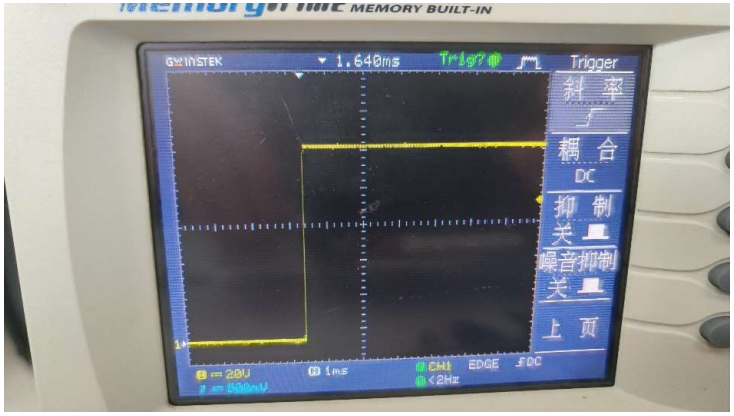
- 可在程序中增加对各次按键中最快按键响应时间的记录与显示，以及平均按键响应时间的检测记录与显示。以便评估你所修改的程序“在能避免误计按键次数的前提下，能对多快的按键速度正确响应并计数”。

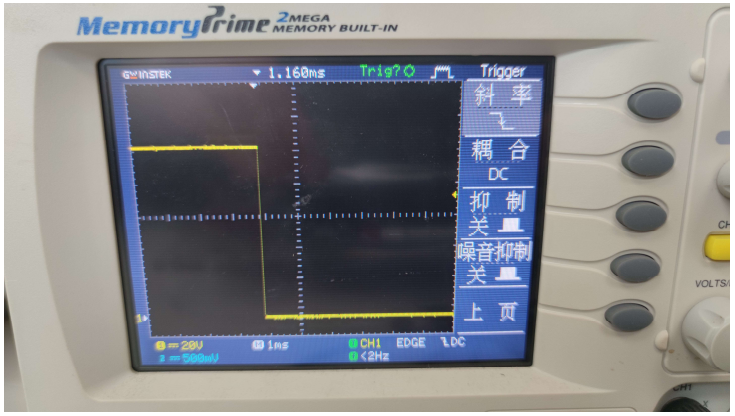
在总结报告中可附上你的这些测试屏显内容截图和必要的测试过程说明。 ⑿-甲5

软硬件联调任务四提示

- 这是一个需要将软硬件相结合的调试问题。
- 如果你观察按键输入口的信号波形——

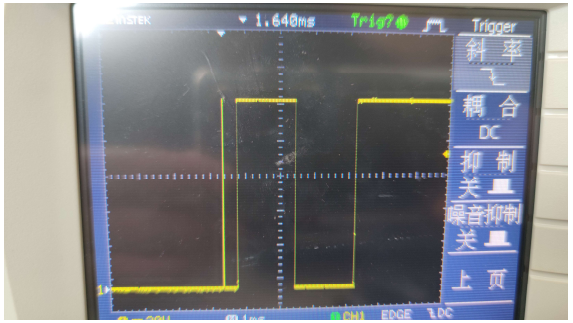
理想的波形如下：



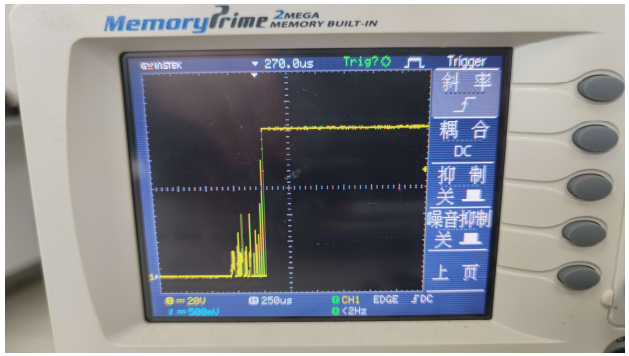


软硬件联调任务四提示

- 实际中更可能是这样的：存在毛刺波形。



- 甚至可能是如下左图那样，展开如右图：





软硬件联调任务四提示

- 这些毛刺由于按键的机械特性而存在。其危害是造成键被误按——一次变多次！
- 常规的去除是在产生毛刺其间暂停对按键信号的读入，这可以用恰当的软件延时来实现。
- 但软件延时过量又会造成对按键响应的迟钝。
- 调试软件使其既能避免毛刺，又能对按键由恰当的及时正确的反应，是一个好键盘的终极最求，需要在按键系统调试过程中结合硬件特性，对软件仔细调教。

（甲）教学班内容

说明：关于代码的提交

- 代码存档：完成上述各项任务的程序代码修改并调试运行，确认测试正确后，将代码（即扩展名为.ino的代码文件改名为.txt后）上传“学在浙大”（注意该文件只有改名成.txt文件才能上传）。文件命名：周x上(或下)午-421(或420)-第x组-任务一(/任务二/任务三/任务四).txt。

另外，总结报告中仅需附上各程序代码片段（不要全部代码，即仅需你修改的部分及其前后一至两行代码即可，报告中附全部代码会有小小的罚分哦！），并在你修改的主要程序语句后标注注释。这些代码片段与本训练项目的其它内容合并于一份报告中一同提交。⑿-甲6

- 再次提醒选做任务二，其测控数据需要写入报告提交。选做任务四，其记录表格、调试时的按键记录波形和原因分析等也需要写入报告提交。

重要提醒

本课件中标注 ✎ 处的实验要求，为你必须记录并写入最终报告的内容！总结报告在本项目结束后再提交！！！